# CACHE DETECTIVE

*Investigando o impacto da hierarquia de memória no desempenho computacional*

| | |
|---|---|
| **Disciplina** | Infraestrutura de Hardware, Bacharelado em Ciência da Computação (3º período) |
| **Aula** | 12: Hierarquia de Memória e Memória Cache |
| **Professor** | Ronierison Maciel |
| **Valor** | **1,0 ponto** |
| **Formato** | Individual ou em dupla |
| **Duração estimada** | 55 minutos |
| **Ambiente** | Navegador + conta no GitHub (**Codespaces**, nada a instalar) ou Linux/WSL2/Docker próprio. Veja `como_rodar.pdf` |

---

## 1. O caso

**Chamado #4721 (prioridade alta)**

A equipe de backend da *DataFlux* tem dois serviços que processam **os mesmos dados**, com **o mesmo número de operações**, na mesma máquina. Um deles é várias vezes mais lento que o outro. O código foi revisado e não há nenhum cálculo a mais. A suspeita da equipe: *"a diferença está na **ordem** em que os dados são lidos da memória"*.

Vocês foram chamados como **analistas de desempenho**. A missão é investigar, com instrumentos adequados, se o **padrão de acesso à memória** explica a diferença, e entregar um parecer técnico.

---

## 2. Os instrumentos: quanto, onde e por quê

Uma investigação de desempenho bem feita responde a três perguntas, e cada uma pede um instrumento diferente:

| Etapa | Pergunta | Instrumento | Tipo de instrumento | Relação com a aula |
|---|---|---|---|---|
| **1** | **Quanto** custa cada padrão? | Cronômetro dentro do programa C (`clock_gettime`) | **Medição real** | Degraus (slides 9 a 12) |
| **2** | **Onde** o programa gasta o tempo? | **gprof** | **Profiling** por amostragem | Não se aplica |
| **3** | **Por quê**? | **Valgrind / Cachegrind** | **Simulação** de cache executando o seu programa | Linha, hit/miss, AoS × SoA (slides 6 a 7, 13 a 14) |

**Atenção às diferenças:**

- **Medição** (Etapa 1): mostra o tempo real no **seu** computador. Esse tempo varia entre computadores e até entre execuções.
- **Profiling** (Etapa 2): o gprof interrompe o programa a cada 10 ms e anota em qual função ele estava. É uma **estimativa estatística** de onde o tempo é gasto. Ele **não** diz nada sobre a cache.
- **Simulação** (Etapa 3): o Cachegrind executa o seu programa instrução por instrução e passa cada endereço lido por uma **cache imaginária**, configurada por nós. Ele **não** mede a L1, a L2 ou a L3 físicas do seu processador.
- **Observação física** da cache, por contadores de hardware (`perf stat`, slide 13), só funciona em Linux nativo e é **opcional** nesta atividade.

### O programa investigado

O arquivo `codigo/cache_detective.c` contém os "dois serviços" da DataFlux:

| Função | O que faz |
|---|---|
| `perseguir()` | Percorre um vetor seguindo ponteiros, em ordem **sequencial** ou **aleatória**, para medir a latência de cada nível da hierarquia (o experimento dos degraus) |
| `soma_linhas()` × `soma_colunas()` | Somam **a mesma matriz** (guardada por linhas, como em C), percorrendo por linhas ou por colunas |
| `soma_aos()` × `soma_soa()` | Somam o campo `x` **das mesmas partículas**: a struct de 64 bytes do "Ache o bug" (AoS) e a versão com um vetor só para os `x` (SoA) |

Leiam essas funções antes de começar. Elas são curtas e comentadas.

---

## 3. Roteiro

| Etapa | O que fazer | Tempo |
|---|---|---|
| 0 | Abrir o ambiente (Codespaces) e, enquanto ele prepara, ler o caso e o código | 5 min |
| | Formulação das hipóteses (**antes** de rodar) | 5 min |
| 1 | Medição de tempo (`make medir`) | 10 min |
| 2 | Profiling (`make gprof`) | 6 min |
| 3 | Simulação de cache (`make cachegrind`, 3 rodadas) | 9 min |
| | Análise e interpretação dos resultados | 10 min |
| | Respostas às questões e entrega | 10 min |
| | **Total** | **55 min** |

Todos os comandos são executados **dentro da pasta `codigo`**, no terminal Linux (Codespaces, WSL2, Linux nativo ou Docker). Como abrir o ambiente: `como_rodar.pdf`. Registrem tudo no **`modelo_entrega.md`**.

> **Comecem pelo ambiente:** abram o link do Codespaces (`como_rodar.pdf`, Caminho A) **antes** de ler o caso. Ele leva de 3 a 5 minutos para ficar pronto e prepara tudo sozinho.

---

## 4. Hipóteses (antes de executar!)

Errar a hipótese **não** tira nota. Deixar de registrá-la ou de justificá-la, **sim**.

**H1: Degraus.** O programa faz **sempre o mesmo número de acessos** em vetores de 4 KB até 128 MB. Para cada percurso, qual será o formato do gráfico *tempo por acesso* × *tamanho*?

| |
|---|
| Percurso **sequencial** |
| Percurso **aleatório** |

**H2: Conta de misses.** Considerem uma cache com linhas de **64 bytes** que começa vazia (a configuração da Etapa 3).

1. Uma matriz de **1000 × 1000 ints** (4 bytes cada), guardada por linhas, é percorrida **por linhas**. Quantos misses vocês esperam? E percorrendo **por colunas**, se uma coluna inteira não couber na cache?
2. São **200.000 partículas**. Na versão **AoS** (struct de 64 bytes), quantos misses o laço que lê só `p[i].x` vai causar? E na versão **SoA** (vetor só de `x`, 8 bytes cada)?

**H3: Profiling.** Qual função vocês esperam que **domine** o tempo no relatório do gprof? Justifiquem em uma frase.

---

## 5. Etapa 1: Quanto custa? (medição real) · ~10 min

```
make info      # processador e caches do seu computador
make medir     # compila e executa as duas medições (20 s a 1 min)
```

`make medir` executa dois modos do programa:

- **degraus**: latência em nanossegundos (ns) por acesso, de 4 KB a 128 MB, com 5 repetições e mediana. Gera `resultados/degraus.csv` e o gráfico `resultados/degraus.svg`. Para abrir o gráfico: no Codespaces, clique com o botão direito em `degraus.svg` → **Open With…** → **Image Preview**; no WSL, `explorer.exe resultados/degraus.svg`; nos outros casos, abram o arquivo no navegador.
- **padroes**: tempo das quatro somas, com 7 repetições, mostrando **mediana, mínimo e máximo**, e as razões *colunas* ÷ *linhas* e *AoS* ÷ *SoA*.

Em seguida, **rodem as somas uma segunda vez** e comparem com a primeira:

```
make padroes
```

**Registrem** (Tabelas 1 e 2 do modelo de entrega):

- a saída de `make info` e o ambiente usado (Codespaces, WSL2, Linux nativo ou Docker);
- o gráfico dos degraus e os **patamares** que vocês identificarem (faixa de tamanhos e latência aproximada de cada um);
- as duas execuções de `padroes` (mediana, mínimo e máximo de cada função).

> Para medições mais confiáveis: deixem o notebook **na tomada**, fechem programas pesados e não usem o computador enquanto o programa roda.

---

## 6. Etapa 2: Onde o tempo é gasto? (gprof) · ~6 min

```
make gprof
```

O Makefile compila uma versão instrumentada ( `-pg` ), executa as somas 50 vezes e gera o **perfil plano** (*flat profile*) em `resultados/gprof.txt` .

As colunas mais importantes do relatório:

| Coluna | Significado |
|---|---|
| `% time` | Porcentagem do tempo total gasto **dentro** da função |
| `self seconds` | Segundos gastos dentro da função (cada amostra vale 0,01 s) |
| `calls` | Quantas vezes a função foi chamada |

**Registrem** na **Tabela 3** o `% time` e o `self seconds` das quatro funções `soma_*` .

---

## 7. Etapa 3: Por quê? (Cachegrind) · ~9 min

O Cachegrind executa o modo `perfil` do programa (dados menores: matriz 1000 × 1000 e 200.000 partículas, porque a simulação deixa o programa 20 a 50 vezes mais lento). Ele conta, **por função** e **por linha do código**:

| Sigla | Significado |
|---|---|
| `Dr` | *Data reads*: leituras de dados da memória |
| `D1mr` | Leituras que deram **miss na cache L1 de dados** simulada |
| `DLmr` | Leituras que deram miss **também no último nível** (LL), ou seja, foram até a RAM |

**Taxa de miss da L1 = D1mr ÷ Dr.**

Façam **três rodadas**, mudando **um parâmetro de cada vez** (formato: `tamanho_bytes,vias,linha_bytes` ):

```
make cachegrind                    # Rodada A: L1 de 32 KB, linha de 64 B (padrao)
make cachegrind D1=65536,8,64      # Rodada B: L1 de 64 KB (o dobro da capacidade)
make cachegrind D1=32768,8,128     # Rodada C: linha de 128 B (o dobro da linha)
```

Cada rodada mostra a tabela por função e grava os arquivos `resultados/cachegrind_D1-<configuracao>_funcoes.txt` (por função) e `..._linhas.txt` (por linha do código).

**Registrem** na **Tabela 4** o `Dr` e o `D1mr` das quatro funções `soma_*` nas três rodadas, e calculem as taxas de miss.

Vale a pena abrir um arquivo `_linhas.txt` e procurar a linha `s += m[i * n + j];`. O Cachegrind mostra exatamente qual linha do código causou os misses.

### Opcional (não vale nota extra)

- **Linux nativo com `perf` instalado:** `make perf` mostra contadores de **hardware reais** de cache. Em geral não funciona no Codespaces, no WSL2 nem no Docker.
- Vetores maiores nos degraus: `make degraus MAX_MB=512`, se houver memória RAM disponível.

---

## 8. Questões investigativas

Respondam com base nos **seus** resultados, citando números das tabelas. Cada resposta deve ter de **3 a 8 linhas**.

**Q1: Lendo os degraus.** Quantos patamares vocês identificaram na curva do percurso **aleatório**? Para cada um, informem a faixa de tamanhos e a latência aproximada. Comparem os pontos de mudança com os tamanhos de cache de `make info` e as latências com a tabela do slide 5. O que cada salto revela sobre a hierarquia de memória?

**Q2: O sequencial não sobe os degraus.** Com 128 MB, o vetor é muito maior que qualquer cache, e mesmo assim o percurso **sequencial** custa poucos nanossegundos por acesso. Expliquem esse comportamento usando **linha de cache** e **localidade espacial**. Por que o percurso aleatório não tem a mesma vantagem, se lê exatamente os mesmos elementos?

**Q3: Medir não é rodar uma vez.** Comparem as duas execuções de `make padroes`. (a) As medianas mudaram muito? E os máximos? (b) Por que o programa repete cada medição e informa a **mediana**, e não a média nem uma única execução? (c) Em um artigo científico, quais números vocês reportariam e por quê?

**Q4: Quanto × onde × por quê. As três etapas concordam?** Montem a **Tabela 5** do modelo de entrega com três razões para cada par de funções: (i) tempo da Etapa 1, (ii) `self seconds` do gprof, (iii) `D1mr` da Rodada A. (a) As três etapas apontam o mesmo "culpado"? (b) **A razão de misses explica a razão de tempo?** (slide 15, item 3). Se os números não são iguais, apresentem pelo menos dois motivos. Dica: olhem a coluna `DLmr` da matriz por colunas. (c) O que o gprof **não** consegue dizer sobre o problema, e que só o Cachegrind mostrou?

**Q5: Mudando a cache simulada.** (a) Na Rodada B (L1 de 64 KB), qual função mudou bastante e quais não mudaram? Expliquem usando a quantidade de linhas de cache que **uma coluna** da matriz ocupa. (b) Na Rodada C (linha de 128 B), o que aconteceu com `soma_linhas`, `soma_soa` e `soma_aos`? Por que a AoS deixou de ter 100% de miss? (c) O Cachegrind mediu a cache do seu processador? Justifiquem.

**Q6: Parecer técnico e metodologia.** (a) Escrevam um parecer de 5 a 8 linhas para a equipe do chamado #4721: o padrão de acesso explica a diferença? Citem **uma evidência de cada etapa** e deem **duas recomendações práticas** de programação. (b) Listem **quatro informações** que vocês declarariam na seção *Metodologia* de um artigo para que outra pessoa pudesse reproduzir estas medições.

---

## 9. Entrega

**Prazo: até o final da aula, aqui no Classroom, com o que vocês tiverem conseguido fazer.** Entrega parcial vale nota parcial: é melhor entregar as etapas que rodaram, bem interpretadas, do que não entregar.

**Um único arquivo (PDF ou Markdown) por dupla ou estudante**, preferencialmente a partir do `modelo_entrega.md`, contendo:

1. Nome dos integrantes.
2. Hipóteses H1, H2 e H3, com justificativa.
3. Breve descrição do que foi executado (2 a 4 linhas) e do ambiente (`make info` e Codespaces/WSL2/Linux/Docker).
4. Resultados: Tabelas 1 a 5, com o gráfico dos degraus e ao menos uma captura de tela do gprof e uma do Cachegrind.
5. Respostas às questões Q1 a Q6.
6. Conclusão breve (3 a 5 linhas) relacionando os resultados à teoria da aula.

**Anexo:** a pasta `resultados/` compactada. Rodem `zip -r resultados.zip resultados`. No Codespaces, cliquem com o botão direito em `resultados.zip` → **Download...**; no WSL, use `explorer.exe .`. Se alterarem o código, anexem o `.c` e expliquem o que mudou.

**Não é necessário:** capa, formatação ABNT ou referências bibliográficas extensas.

### Se o ambiente não funcionar

1. Consultem a seção **Problemas comuns** de `como_rodar.pdf`.
2. Tentem outro caminho (Codespaces, Linux/WSL2/Docker próprio) ou façam em dupla com um colega cujo ambiente funcionou, registrando qual máquina foi usada.
3. Em último caso, usem os **dados de referência fornecidos pelo professor** (`dados_referencia/`). Escrevam na entrega: *"Etapas X e Y: dados de referência fornecidos pelo professor (Apple M1), pois [motivo]"*. A interpretação vale a mesma nota. O que não pode é apresentar dados de referência como se fossem seus.

---

## 10. Como a atividade será avaliada (1,0 ponto)

| Critério | O que será observado | Valor |
|---|---|---|
| Hipóteses e compreensão do problema | H1, H2 e H3 registradas **antes** da execução, com justificativa e cálculo de H2 com raciocínio | 0,15 |
| Execução e registro das evidências | Ambiente identificado; Tabelas 1 a 4 completas; gráfico e capturas; números coerentes com os arquivos anexados | 0,25 |
| Interpretação dos resultados | Q1, Q3, Q4 e Q5 explicadas com os **seus** números, e não com definições genéricas | 0,30 |
| Relação com os conceitos da aula | Uso correto de localidade, linha de cache, hit/miss e hierarquia, e distinção entre medição, profiling e simulação (Q2, Q6) | 0,20 |
| Conclusão e organização | Conclusão ligada às evidências; entrega organizada e legível | 0,10 |
| **Total** | | **1,00** |

**Seus tempos não precisam ser iguais aos de colegas ou de qualquer exemplo.** Computadores e ambientes (Codespaces, WSL2, Docker, Linux nativo) produzem números diferentes. Será avaliada a **coerência** entre os seus dados e a sua interpretação.

Bom trabalho, detetives!